

BENTO: Fine-Grained Memory Isolation for COTS WebAssembly Binaries

Oussama Draissi

oussama.draissi@uni-due.de
paluno - the Ruhr Institute for Software Technology
University of Duisburg-Essen
Essen, Germany

Lucas Davi

lucas.davi@uni-due.de
paluno - the Ruhr Institute for Software Technology
University of Duisburg-Essen
Essen, Germany

Abstract

WebAssembly (Wasm) enables execution of memory-unsafe languages like C/C++ in the browser, but its monolithic linear memory leaves modules vulnerable to high-impact attacks such as cross-site scripting. Existing defenses are unsuitable for commercial-off-the-shelf (COTS) binaries, requiring either source code, non-standard runtimes, or introduce high-overhead instrumentation.

We present Bento, the first static binary rewriter that automatically hardens COTS Wasm modules. Bento uses whole-program pointer analysis to partition memory into logical regions (stack, heap, globals) and rewrites the binary to map each region to an isolated instance using the standard Wasm multi-memory feature. This creates a software-based MMU that provides architectural hardening without adding instructions or costly runtime checks.

Our evaluation shows that Bento mitigates real-world exploits in prominent applications, including Libpng and PDFAlto, while preserving program semantics, as confirmed by differential testing. Since our architectural approach adds no new instructions, Bento only induces 3% runtime slowdown and a 1% startup overhead.

CCS Concepts

• Security and privacy → Web application security.

Keywords

WebAssembly, Memory Corruption, Sandboxing, Software Fault Isolation, Architectural Hardening

ACM Reference Format:

Oussama Draissi and Lucas Davi. 2026. BENTO: Fine-Grained Memory Isolation for COTS WebAssembly Binaries. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792439>

1 Introduction

WebAssembly (Wasm) [22] has become a foundational technology for the modern web, enabling large, performance-critical codebases to be deployed directly in the browser. Its adoption is accelerating; Wasm instantiation now accounts for over 5.2% of all page loads [20], powering major applications like Figma, Adobe Photoshop, and Google Earth. This growth poses a significant security challenge

because the ecosystem is dominated by code originally written in memory-unsafe languages like C/C++ [26, 28].

The reliance on memory-unsafe languages introduces a critical threat: exploits within a Wasm module can be escalated to attack the surrounding web application [12, 35, 42] because the host environment implicitly trusts data returned from the Wasm sandbox. For instance, a memory corruption bug can enable a data-only attack [30, 31], allowing an adversary to overwrite a trusted data structure with a malicious payload. When the host browser renders this corrupted data, the attack escapes the sandbox leading to a full compromise of the web application by means of Cross-Site Scripting (XSS). The escalation is possible because the Wasm sandbox, while isolating the module from the host, does not enforce memory safety within the module's own monolithic linear memory. This memory co-locates the stack, heap, and global data, allowing a vulnerability in one region to corrupt data in another region. A recent study confirms that such *cross-region exploits* are an emerging attack vector affecting thousands of websites [12].

Protecting Wasm modules in a practical web environment is an open challenge. General-purpose runtime defenses are insufficient or cannot be deployed in practice. A prominent example is address space layout randomization (ASLR) [57] which is incompatible to Wasm's single, monolithic linear memory model. Similarly, Wasm's built-in control-flow integrity (CFI) [1, 9, 59, 70] prevents control-flow attacks but can be bypassed by means of data-only attacks which are the primary source of exploits in Wasm [12, 35, 42]. Although techniques for finding vulnerabilities a priori—such as fuzzing and static analysis [3, 5, 12, 23–25, 36, 67, 73]—are essential, they offer no runtime protection when a user executes a vulnerable module.

This has led to several lines of Wasm-specific defenses, yet none are practical for securing Commercial-off-the-shelf (COTS) binaries that dominate the ecosystem. The majority of proposed systems are incompatible with the web platform, requiring source-level modifications, specialized hardware, or custom runtimes and compilers [33, 34, 37, 44, 46]. While binary rewriting is a more practical, well-established strategy for COTS binaries [1, 8, 9, 11, 13, 18, 27, 41, 47, 52, 53, 59, 70, 71], existing Wasm defenses only apply it for reactive bug detection, i.e., they instrument binaries with runtime checks that match pre-defined bug patterns [36, 67]. This pattern-matching methodology is fragile as its checks target specific syntactic patterns that are often invalidated by compiler optimizations. Further, it incurs high performance overhead, and requires debug symbols which are ineffective for stripped COTS binaries.

In this paper, we envision a more robust security primitive that proactively establishes a strong containment boundary for any



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
WWW '26, Dubai, United Arab Emirates.

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792439>

memory error. To satisfy the pressing requirement on web compatibility and the handling of COTS binaries demands a new *architectural hardening* approach—one that enforces memory error isolation by embedding its protections directly into the Wasm module.

To do so, we present BENTO, the first static analysis and binary rewriting framework that automatically hardens any COTS Wasm module against data-only attacks [12, 35, 42]. At its core, BENTO performs a static analysis to partition a module’s monolithic address space according to two architectural hardening policies. The BENTO-Coarse policy establishes inter-region integrity by partitioning memory into stack, heap, and data regions. This mitigates cross-region exploits: e.g., preventing a stack-based overflow from corrupting DOM-bound data (e.g., HTML tags) in the data region. For stronger, inter-procedural protection, the BENTO-Fine policy further isolates each function’s stack frame, preventing a buffer overflow in a callee from corrupting its caller. This stronger enforcement model introduces a modest trade-off, increasing file size by an average of 2% over BENTO-Coarse.

Both policies enforce these isolation boundaries by rewriting the binary to use the standard Wasm multi-memory feature [62], effectively creating a software-based MMU. In essence, BENTO achieves the following properties: (1) *high efficiency* hardware-like memory isolation without dependencies on specialized features like Intel MPK [37, 43]; (2) *non-invasive protection*, injecting zero new instructions, unlike existing instrumentation-based approaches [36, 67]; and (3) *runtime-agnostic defense*, ensuring compatibility with any standard Wasm runtime [33, 34, 37, 44, 46].

Our evaluation, including the standard WABench [61] test suite and widely-used real-world programs, demonstrates that BENTO provides robust security guarantees while preserving program correctness and incurring negligible overhead. We demonstrate that BENTO mitigates exploits for three real-world vulnerabilities: a stack-based buffer overflow in Libpng (CVE-2018-14550), a heap-based buffer overflow in PDFAlto (CVE-2022-32324), and the Heartbleed information leak (CVE-2014-0160). Comprehensive differential testing confirms our analysis is semantically sound across applications compiled with multiple optimization levels. Unlike instrumentation-based defenses [36, 67], BENTO’s non-invasive approach results in a low average runtime overhead of 3 to 4%, a negligible 1% increase in peak memory and startup overhead, and, critically, **zero computational overhead**, as the number of executed instructions remains identical. Hence, our architectural hardening is a practical and efficient defense for COTS Wasm modules. In summary, we make the following contributions:

- The design and implementation of BENTO, a binary rewriting framework that enforces memory isolation in Wasm by repurposing the multi-memory feature.
- A whole-program pointer analysis for stripped COTS Wasm binaries that robustly identifies pointer origins (e.g., stack, heap, data) without debug symbols.
- An evaluation showing the mitigation of real-world vulnerabilities in widely-used libraries, including Libpng (CVE-2018-14550) and PDFAlto (CVE-2022-32324).
- An analysis showing our architectural hardening approach is highly efficient, providing security with negligible runtime overhead.

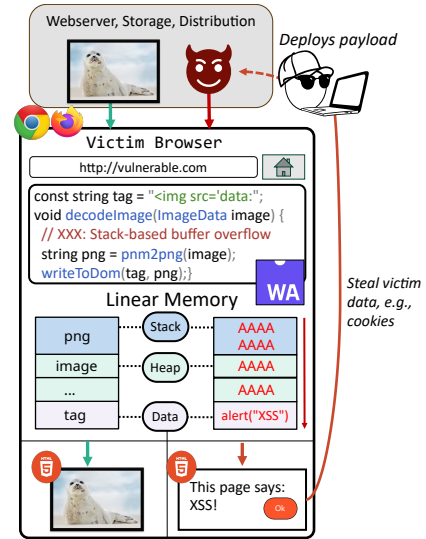


Figure 1: An attacker-provided image triggers a buffer overflow in linear memory, allowing untrusted data to overwrite a trusted HTML string. The host browser renders this corrupted string, leading to an XSS attack.

2 WebAssembly Security

Wasm’s security model [63] is built on three principles: software fault isolation (SFI) [60], control flow integrity (CFI) [1], and code immutability. These mechanisms create a strong sandbox that isolates the Wasm module from the host browser. This prevents code-reuse [4, 10, 55] and code-injection attacks [48].

The Monolithic Linear Memory. While Wasm’s security model effectively protects the host, it lacks equivalent intra-component protection. Central to a Wasm module’s execution environment is its *linear memory*: a single, contiguous, and resizable array of bytes that is the only memory region directly addressable by Wasm instructions. This memory model is *dense*, with no separation between data regions, and implements a *flat permission model*: any instruction can freely read or write any address within the module’s bounds. As a result, this monolithic address space co-locates all of a module’s data—including the stack, heap, and host-provided data—without any runtime-enforced separation between them.

To enable more sophisticated memory architectures, the Wasm specification includes the *multi-memory feature* [62]. This extension allows a single Wasm module to declare and access multiple, independent linear memory instances. Each memory is assigned a unique, static index, and instructions are explicitly bound to a specific memory at compile time. Crucially, the Wasm runtime guarantees that these memories are completely isolated. The static memory index associated with an instruction cannot be altered by the guest code because the code is immutable. Notably, while high-level languages do not yet provide abstractions for this feature [58], it provides the foundational primitive for our approach to transform the Wasm runtime into a software-based MMU.

Memory Corruption and The Web-Wasm Threat Model. Despite Wasm’s strong protections, vulnerabilities from memory-unsafe

Table 1: Comparison of Wasm security frameworks across properties critical for practical web deployment.

System	(1)	(2)	(3)	(4)	(5)
PKUWA [37]	✗	✗	✗	✓	✗
RLBox [46]	✗	✓	✓	✗	✓
MS-Wasm [34, 44]	✗	✓	✗	✗	✗
Wave [33]	✗	✓	✓	✗	✓
Fuzzm [36] & WBSan [67]	✓	✓	✗	✗	✗
BENTO (ours)	✓	✓	✓	✓	✓
Columns denote: (1) Web Platform Compatible (2) Fully Automatic (3) COTS-compatible (4) Fine-Grained Isolation (5) Non-invasive					

languages persist when compiled to Wasm. The monolithic linear memory model creates an attack surface by co-locating all data in a single, predictable, and fully writable address space. Consequently, a single memory corruption bug allows untrusted input to overwrite trusted application data anywhere in memory. As illustrated in Figure 1, the attack chain begins when a malicious payload processed by a vulnerable Wasm module corrupts data that is passed back across the SFI boundary. The host environment, lacking visibility into the corruption, implicitly trusts this data and may render it directly into the DOM, leading to arbitrary code execution. This enables an attacker to corrupt a wide range of critical targets, including constant data like the trusted HTML string (tag) shown in Figure 1, heap allocator metadata to gain arbitrary write primitives, function pointer indices for control-flow hijacking, and the stack frames of caller functions due to the absence of stack canaries [35].

A recent study [12] confirms the prevalence of this dangerous host-module trust relationship, identifying thousands of domains passing untrusted data to unsafe sinks, for instance, `Function.ctor` and `innerHTML`. While this provides a direct primitive for XSS, the fundamental threat is the violation of data integrity. A single memory corruption vulnerability, for instance, can allow an adversary to subvert the client-side application completely. An attacker could hijack fetch requests to inject malicious payloads or exfiltrate data, and poison `sessionStorage` to establish persistent XSS. By enforcing architectural isolation, BENTO prevents such integrity violations, thereby neutralizing this entire class of threats.

3 Problem Statement

While memory corruption in Wasm is a known threat [12, 35, 42], existing defenses are impractical for web deployment. This impracticality stems from their failure to meet five requirements for a practical defense, as summarized in Table 1. Our analysis identifies two primary classes of limitations: incompatibility with the standard web platform and a reliance on fragile, invasive bug-detection techniques.

Incompatibility with the Web Platform. Many defenses are incompatible with the standard web platform and cannot be used in standard browsers. As shown in Column 1 of Table 1, systems like PKUWA [37], RLBox [46], MS-Wasm [34, 44], and Wave [33] are not web-platform compatible. They require special hardware (e.g., Intel MPK [43]), custom runtimes, or non-standard extensions to

the Wasm specification. Furthermore, approaches like MS-Wasm and PKUWA require access to the application’s source code (Column 3) because they use custom compilers. The requirement for source code is impractical, as web applications heavily rely on pre-compiled, third-party COTS libraries for which source code is unavailable. Finally, some of these systems, like PKUWA, are not fully automatic (Column 2), demanding manual code changes that do not scale to complex software.

Reliance on Fragile and Invasive Bug Detection. A second class of defenses, while web-compatible, relies on reactive bug detection instead of proactive architectural hardening. Systems like Fuzzm [36] and WBSan [67] instrument Wasm binaries to detect specific bug patterns at runtime, an approach with two fundamental weaknesses. First, it is fragile. Both approaches are limited to matching pre-defined syntactic patterns that compiler optimizations can invalidate. For instance, a check matching an addition between two variables ($x + y$) is fragile because it fails to account for operands from other sources, such as compiler-introduced constants ($x + 10$), global variables, or function calls. Second, this approach is highly invasive (Column 5). A practical defense must be non-intrusive because heavyweight instrumentation—adding many checks and branches—imposes significant runtime overhead and increases binary size, which slows down the Wasm module. The effectiveness of these tools is also limited in real-world deployments as they depend on debug symbols to understand the binary’s structure, e.g., to correctly place heap canaries next to memory allocations. Because COTS binaries are stripped of debug symbols, these defenses fail in the exact scenarios where they are mostly needed (Column 3), as they cannot reliably establish memory boundaries (heap canary locations) without symbolic information.

4 Design

To overcome the limitations of prior work, we designed BENTO, a static binary rewriting framework that provides fine-grained memory isolation for WebAssembly. BENTO’s design is guided by five requirements for practical web deployment: it must be ① fully compatible with the standard web platform, ② automatic and scalable, ③ treat black-box COTS modules, ④ provide fine-grained architectural isolation, and ⑤ do so in a non-invasive manner. BENTO meets these requirements through proactive *architectural hardening*, embedding its security policy directly into the binary’s structure rather than injecting runtime checks.

At its core, this architectural hardening is achieved through a precise, whole-program pointer analysis that deconstructs a module’s monolithic linear memory into a set of logically distinct regions. This process enables two security policies with different levels of protection:

- **BENTO-Coarse** partitions memory into stack, data, and heap regions. Its security rationale is to prevent cross-region memory corruption, the central Wasm exploit vector. This policy directly mitigates the entire class of vulnerabilities where, for instance, a stack-based overflow corrupts heap metadata or DOM-bound data.
- **BENTO-Fine** provides stronger, intra-stack containment by creating a separate memory instance for *each* function’s stack frame. Its rationale is to prevent inter-procedural attacks:

a buffer overflow in a callee’s stack frame can no longer corrupt its caller’s frame. The attack is thereby downgraded from a severe vulnerability into a localized data corruption bug.

BENTO enforces these isolation boundaries by rewriting the binary to use the standard Wasm multi-memory feature [62], effectively creating a software-based MMU. This entire process is realized in three main stages, visualized in Figure 2:

- (1) **IR Lifting (Section 4.1):** We lift the Wasm binary into a context-sensitive, interprocedural value-flow graph, the BentoIR.
- (2) **Pointer Origin Analysis (Section 4.2):** We perform a data-flow analysis over the IR to determine the logical origin (*stack*, *data*, or *heap*) of every pointer.
- (3) **Binary Transformation (Section 4.3):** We use the analysis results to rewrite the module, enforcing isolation by mapping regions to separate Wasm memory instances.

4.1 BentoIR: Interprocedural Value-Flow Graph

BENTO first lifts a Wasm binary into a Static Single Assignment (SSA) representation. From this, it constructs the BentoIR, a precise, interprocedural graph of the program’s entire value flow. In this graph, each SSA value is a semantically-typed node, and directed edges represent value-flow dependencies. This semantic typing is central to the precision of our subsequent analyses; for instance, a `LoadNode` explicitly records its address operand, while a `StoreNode` captures both its address and data operands.

The key to our interprocedural precision is *context-sensitive stitching*: for each function call, we instantiate a unique set of parameter and return-value nodes for the callee. As illustrated in Figure 3(b), this ensures that value flows from different call sites remain distinct, preventing the collapse of calling contexts—e.g., ensuring a pointer to the *stack* from one call site is not conflated with a pointer to the *data* region from another. This precision is necessary to prevent incorrect pointer classifications, which would otherwise lead to unsound isolation policies.

4.2 Pointer Origin Analysis

Our analysis computes the set of possible memory origins for every pointer by propagating origin properties through the entire BentoIR. The security purpose of this stage is to define a concrete isolation policy by classifying every pointer based on its logical origin (*stack*, *data*, or *heap*). Because the IR represents the whole program, this propagation naturally crosses function boundaries, tracking origins through both arguments and return values. To handle memory’s implicit data-flow channel, we perform a targeted alias analysis to construct explicit edges for definite store-to-load dependencies.

The analysis is seeded by a set of high-confidence *anchors* that provide ground-truth origin information: *stack pointers*, identified via semantic analysis of the stack pointer global; *data pointers*, derived from constant addresses that fall within the binary’s data segments; and *heap pointers*, derived from functions that exhibit allocator-like semantics. The semantic correctness of these anchors is empirically validated in our evaluation (Section 8.2).

4.3 Binary Transformation

The final stage uses the analysis results to enforce the security policy, rewriting every memory instruction to target the memory instance corresponding to its pointer’s origin. This provides irreversible enforcement, as the target memory index is encoded into each instruction’s immutable bytecode, making the Wasm runtime the enforcer of our isolation policy.

The primary challenge is a *polymorphic* memory access, where an instruction may operate on pointers of different origins. Our analysis precisely identifies these cases (Figure 3(b)). We resolve this ambiguity using **function specialization**: we create specialized clones of the polymorphic function for each origin context and update all call sites to invoke the appropriate version (Figure 3(c)).

This approach provides irreversible enforcement because the target memory index is encoded directly into each memory instruction’s *immutable bytecode*. Consequently, no memory corruption vulnerability within the module can alter which memory an instruction accesses, making the Wasm runtime the ultimate enforcer of our isolation policy.

5 Implementation

We implemented BENTO in 4885 lines of Rust. It uses the waffle library [17] for binary parsing, SSA transformation, and rewriting. The BentoIR is implemented using the petgraph library. The implementation performs the analysis and transformation stages of the pipeline (Figure 2) and is designed to be robust against stripped, COTS binaries.

To operate without debug information, BENTO identifies the analysis anchors from Section 4.2 using symbol-independent heuristics. Because the stack pointer must be shared across functions, it is implemented as a global variable in Wasm. Our analysis semantically identifies it by finding the global variable consistently used as a base for pointer arithmetic and memory accesses, a method independent of any specific ABI. Data pointer anchors are identified as constants validated against the module’s data segment bounds. Finally, our semantic heap anchor identifies allocator functions—including custom memory allocators, as shown in Section 8.2—as any function whose returned pointer cannot be traced to a stack or data anchor. The use of these robust, symbol-independent heuristics is essential for partitioning stripped COTS binaries.

Our implementation of the propagation analysis uses a standard worklist algorithm. The BentoIR’s SSA form makes this process highly efficient, since the origin of any value is a function of its immediate predecessors in the graph. This graph-based approach naturally handles the complex pointer derivations common in optimized COTS binaries, where the computation of an address can span multiple instructions, basic blocks, or even entire functions. For any memory access, our analysis resolves this complexity by simply traversing backwards through the value-flow graph from the SSA value used as the address. This process precisely identifies polymorphic memory accesses—where a single instruction may operate on pointers from different regions—such as the `i32.load` instruction highlighted in Figure 3(b). This graph-based propagation handles arbitrarily complex pointer derivations and makes the analysis both precise and efficient.

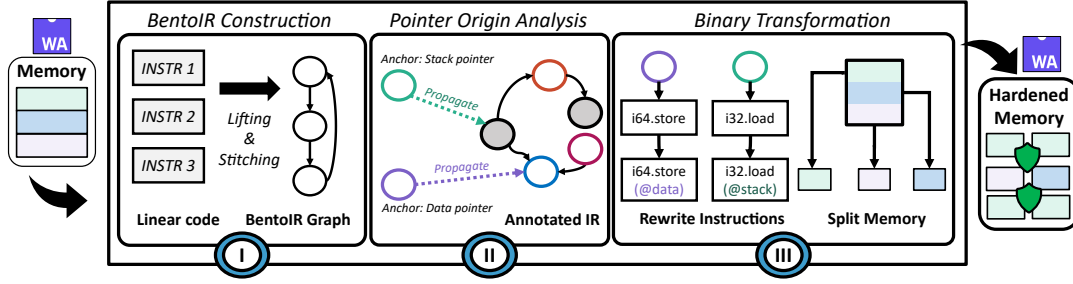


Figure 2: The BENTO analysis and transformation pipeline.

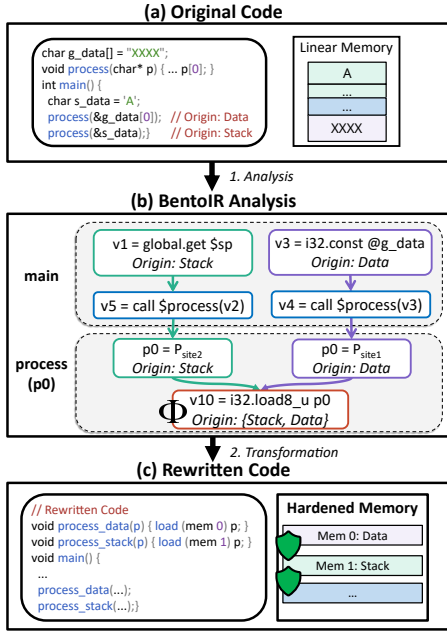


Figure 3: BENTO’s analysis and transformation pipeline. (a) A function process is called with pointers from different origins (*data* and *stack*), creating a polymorphic memory access. (b) Our context-sensitive analysis identifies the polymorphic load instruction (red). (c) The transformation resolves this via function specialization, creating hardened clones with monomorphic accesses to enforce memory isolation.

Another challenge is handling indirect function calls, as their targets are resolved at runtime. Our approach addresses this by conservatively over-approximating the set of potential callees. Wasm’s CFI guarantees that all potential targets of an indirect call are declared in a table and share the same function signature. We leverage this by analyzing all functions in the corresponding table that share the potential callees signature. If the arguments at the call site could introduce polymorphism in any of the potential targets, we create specialized versions of those targets as needed. This robust handling remains efficient, increasing file size by an average of only 4%

for BENTO-Coarse and 6% for BENTO-Fine—a modest overhead compared to the 51% imposed by instrumentation-based approaches like WBSan (Section 8.3).

6 Security Considerations

BENTO’s primary security contribution is to raise the cost and complexity of exploiting memory corruption errors in Wasm. By partitioning the monolithic address space, BENTO systematically mitigates the threats described in the Web-Wasm threat model (see Section 2), making cross-domain exploits [12, 35, 42] infeasible and forcing an adversary to construct significantly more complex, intra-region attacks.

The BENTO-Coarse policy establishes a foundational guarantee of *inter-region memory integrity*. Our partitioning mitigates any exploit that requires crossing a logical memory boundary. For instance, a stack-based buffer overflow cannot be used to corrupt heap allocator metadata to gain an arbitrary write primitive, or from overwriting a function pointer index stored on the heap to hijack control flow [35, 42]. Similarly, BENTO prevents the *spatial* consequences of a use-after-free (UAF): a dangling heap pointer cannot corrupt sensitive data residing on the stack.

The BENTO-Fine policy enforces *inter-procedural data integrity*, preventing exploit chains that cross function call boundaries. In a Wasm module, a buffer overflow in a callee can corrupt its caller’s stack frame [35]. An attacker could leverage this to corrupt sensitive data (e.g., pointers) or overwrite a critical variable in the caller’s scope, compromising the application after the callee returns. BENTO-Fine prevents this by placing each frame in a separate memory instance. The exploit is thus confined to the callee’s context, preserving the caller’s integrity and downgrading a potentially devastating exploit to a localized bug.

Scope and Limitations. BENTO provides vulnerability containment, mitigating the impact of memory errors rather than eliminating or detecting the bugs themselves. While it makes cross-domain exploits infeasible, intra-region and intra-frame corruption remains possible. For example, BENTO does not aim to provide *temporal* safety such as use-after-free exploits that corrupt metadata of an adjacent object within the same heap instance. Likewise, a stack buffer overflow is confined to overwriting local variables within the *same stack frame*, but it cannot escape to corrupt the caller. Weaponizing such localized bugs requires deep, application-specific knowledge of memory layouts and is significantly harder than traditional attacks.

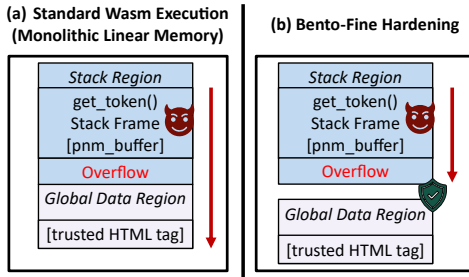


Figure 4: Mitigation of a cross-region stack overflow. (a) In standard Wasm, a stack-based buffer overflow corrupts trusted data in the adjacent global data region. (b) BENTO places the stack and global data in separate, isolated memory instances, containing the overflow within the stack and neutralizing the exploit.

Just as BENTO provides inter-region isolation by separating the stack and heap, this same architectural principle can be applied to provide intra-frame isolation. This extension, however, requires a substantially more granular static analysis capable of robustly deconstructing a function’s stack frame into distinct variable-level regions without source code or debug symbols. While such an analysis is beyond the scope of this paper, it presents a clear and promising direction for future work, building on the architectural foundation we establish here to mitigate even highly localized memory errors.

7 Case Studies

To demonstrate BENTO’s practical security guarantees, we evaluate it against three real-world vulnerabilities previously used to assess Wasm defenses [12, 35–37, 67]. We constructed attack scenarios using a host web page that integrates a vulnerable Wasm module and processes its output. This setup mirrors platforms like Google’s CanvasKit, which assume data returned from Wasm is trustworthy. In this practical browser environment, we show how BENTO’s architectural hardening prevents the escalation of memory corruption into impactful web attacks.

Libpng (CVE-2018-14550). A crafted image triggers a stack-based buffer overflow in Libpng. In an unhardened module, this overflow corrupts a trusted HTML string in the global data region, leading to a cross-region XSS attack (Figure 1). BENTO mitigates this exploit by isolating the stack from the data region. The stack overflow is contained and cannot corrupt the DOM-bound HTML tag, preventing the escalation to an XSS attack (Figure 4). BENTO-Fine provides a stronger guarantee by further confining the bug to the vulnerable function’s stack frame, thereby preserving caller integrity.

PDFAlto (CVE-2022-32324). A heap-based buffer overflow allows an attacker to corrupt data on the stack or in global segments. BENTO prevents this by isolating the heap in a dedicated memory instance, containing the overflow and protecting all other memory regions.

Heartbleed (CVE-2014-0160). The classic Heartbleed vulnerability is an out-of-bounds read that leaks sensitive data, such as private

keys, from other memory regions. BENTO strictly confines this information leak to the heap region where the buffer resides, preventing the read from accessing any data on the stack or in global segments.

8 Evaluation

Our evaluation shows that BENTO provides robust memory isolation for Wasm while preserving program correctness and incurring negligible performance overhead, making it practical for web deployment. We structure our analysis around three key questions regarding correctness and applicability, static overhead, and runtime overhead.

8.1 Methodology

All experiments were conducted on a machine with an AMD EPYC 7302P Processor running Ubuntu 24.04. We compiled all benchmarks to Wasm using Zig (v0.16.0), except for the AddressSanitizer (ASan) [54] baseline which required Emscripten (v4.0.14). All Wasm modules were evaluated in the Wasmtime runtime (v36.0.2) to obtain precise and repeatable performance measurements that are not possible in noisy browser environments. To mitigate transient effects from JIT compilation and system caches, we perform three dedicated warmup runs for each benchmark and report the average of 20 subsequent measured executions. Because BENTO’s hardening is architectural, its security guarantees are enforced by any standards-compliant runtime.

We evaluate our system using a set of 52 applications, combining the standard WABench test suite [61] with other real-world programs previously used to evaluate Wasm defenses [36, 37, 67]. This suite is representative of common Wasm use cases and includes applications from scientific computing (PolyBench [50] and JetStream2 [32]), cryptography (e.g., libtomcrypt [40]), image processing (e.g., facedetection [38]), and natural language processing (espeak [16]). Appendix A provides a complete list of benchmarks and their workloads. Our evaluation compares four configurations against an unmodified Wasm binary:

- **Bento-Coarse and Bento-Fine**
- **WBSan [67]:** WBSan requires debug symbols and failed to process any of the stripped binaries in our evaluation. To establish a performance baseline, we recompiled our benchmarks with debug symbols enabled, a configuration that is unrealistic for COTS deployments.
- **ASan [54]:** A heavyweight memory safety tool used to establish a practical upper bound for instrumentation overhead.

We exclude several systems from our quantitative comparison. PKUWA [37] is not fully automatic, as it requires manual source code modifications for each program. Furthermore, MS-Wasm’s [34, 44], Wave’s [33], and RLBox’s [46] technical designs prevent a fair comparison of key metrics. For instance, the use of AOT compilation, makes it impossible to measure JIT instantiation time—a critical metric for web performance.

8.2 Correctness and Applicability

A foundational requirement for any binary rewriting framework is the preservation of program semantics [1, 8, 9, 11, 18, 27, 41, 47, 52, 71], which we validated for BENTO through a comprehensive differential testing campaign. Our methodology involves generating

Table 2: Comprehensive overheads relative to an uninstrumented Wasm module. BENTO consistently demonstrates the lowest overhead. Cells are colored green for improvements and red for degradations. ‘–’ denotes missing data.

Workload	Bento-Coarse		Bento-Fine		WBSan		ASan		Workload	Bento-Coarse		Bento-Fine		WBSan		ASan	
	Time	Fuel	Time	Fuel	Time	Fuel	Time	Fuel		Peak RSS	Inst. Time	Peak RSS	Inst. Time	Peak RSS	Inst. Time	Peak RSS	Inst. Time
Overall	1.03	1.00	1.04	1.00	1.54	1.50	2.42	2.12	Overall	1.01	1.01	1.01	1.00	1.01	1.02	2.49	1.81
polybench	1.05	1.00	1.05	1.00	1.47	1.49	1.99	1.19	polybench	1.00	1.00	1.00	0.99	1.01	1.01	2.22	1.62
mibench	1.02	1.00	1.03	1.00	1.52	1.50	1.93	1.30	mibench	1.03	1.09	1.03	1.05	1.02	1.02	2.92	2.45
jetstream	1.01	1.00	1.03	1.00	1.48	1.48	2.87	1.19	jetstream	1.01	0.97	1.01	0.97	1.02	1.00	5.08	1.67
abc2mtex	0.99	1.00	1.09	1.00	1.66	1.52	8.72	62.09	abc2mtex	1.00	1.03	1.00	0.98	1.01	1.02	2.17	1.80
bzip2	0.98	1.00	1.01	1.00	2.30	1.49	—	—	bzip2	1.00	0.96	1.00	1.00	1.01	1.06	—	—
espeak	1.00	1.00	1.00	1.00	1.70	1.53	—	—	espeak	1.00	1.03	1.01	1.02	1.02	1.03	—	—
facetedetection	0.98	1.00	0.99	1.00	1.40	1.53	—	—	facetedetection	1.00	0.96	1.00	0.96	1.01	1.24	—	—
gnuchess	1.00	1.00	0.99	1.00	1.53	1.56	—	—	gnuchess	1.00	1.00	1.00	1.02	1.01	0.94	—	—
hashsum	1.06	1.00	1.13	1.00	2.03	1.50	—	—	hashsum	1.00	1.01	1.00	0.98	1.02	1.07	—	—
lrcrypt	1.00	1.00	1.00	1.00	2.25	1.43	—	—	lrcrypt	1.01	1.04	1.01	1.02	1.01	1.05	—	—
mnist	1.00	1.00	1.00	1.00	1.39	1.47	2.59	2.12	mnist	1.00	0.98	1.00	1.13	1.00	1.16	1.69	1.92
pdfresurrect	0.99	1.00	1.06	1.00	1.66	1.50	16.71	4023.99	pdfresurrect	1.00	1.05	1.00	0.90	1.00	1.01	2.43	1.92
pnm2png	1.06	1.00	1.09	1.00	1.84	1.55	13.85	13784.44	pnm2png	1.00	0.98	1.00	0.97	1.01	1.05	2.31	1.83
snappy	1.00	1.00	1.00	1.00	1.54	1.55	4.94	2.41	snappy	1.00	1.03	1.00	0.96	1.00	1.09	1.20	2.05
http-parser	1.02	1.00	1.02	1.00	2.57	1.54	18.82	382.96	http-parser	1.01	0.89	1.01	0.90	1.01	0.96	2.70	1.93
whitedb	0.99	1.00	0.99	1.00	1.56	1.50	—	—	whitedb	1.00	1.06	1.00	1.12	1.00	1.16	—	—

a reference execution trace from an unmodified binary by logging every memory access and comparing it against identical logs from the BENTO-hardened variants.

Our base evaluation corpus consists of our 52 core benchmarks, 300 diverse programs generated with csmith [68], and the three real-world applications with known vulnerabilities from our case studies (Section 7), for a total of 355 unique applications. To rigorously stress-test our analysis against a wide variety of code structures, we compiled each of these 355 applications without debug symbols at three different optimization levels: -O0, -O2, and -O3. This is a critical step, as optimizations significantly alter code generation, particularly in function prologues and stack usage, resulting in structurally diverse BentoIRs for the same source program. Our total evaluation set thus comprises 1065 unique binaries.

Across all binaries, spanning all applications and optimization levels, the memory access traces produced by both BENTO variants were 100% identical to their respective baseline traces. This result confirms that BENTO’s rewriting is semantically sound and that our analysis is robust to the significant structural variations introduced by compiler optimizations. Furthermore, we successfully analyzed and hardened 100% of the corpus without any manual intervention. Notably, our semantic analysis (Section 4.1) was effective on real-world code, even correctly identifying custom memory allocators (e.g., in our Libpng case study). This demonstrates that BENTO is not only correct but, crucially, applicable to the stripped COTS binaries where approaches requiring debug symbols [36, 67] or source code [34, 37, 44] fail.

8.3 Static and Startup Overhead

In addition to correctness, a practical hardening approach must be non-intrusive [27, 41, 52, 53], minimizing its impact on binary size and startup performance, two critical factors for web applications.

As illustrated in Figure 5, BENTO’s design is fundamentally non-invasive, with comprehensive data provided in Table 5 in the appendix. Its architectural hardening adds **zero new instructions or branches** to existing function bodies, a property confirmed by the zero injected instructions and branches across all workloads. The

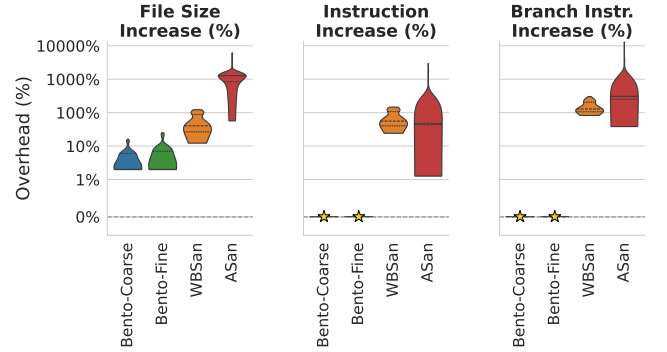


Figure 5: Distribution of static overheads relative to an uninstrumented Wasm module. The violin plots show the overhead distribution across our benchmarks on a logarithmic scale. Note that both BENTO variants add zero new instructions or branches to functions.

system’s only modification to the code section is function specialization, which results in a modest average file size increase of 4% for BENTO-Coarse and 6% for BENTO-Fine. While for some workloads the increase is more pronounced—such as snappy, which grows by 23% under BENTO-Fine—this overhead remains lower than the 51% average file size increase imposed by instrumentation-based defenses like WBSan.

Crucially, this low code overhead ensures that startup performance is not degraded. As shown in Table 2, BENTO adds negligible overhead to module instantiation time (<1% on average), even for binaries with a larger file size. This non-intrusiveness also extends to memory footprint. Peak Resident Set Size (RSS) is a critical metric for web applications, and as shown in Table 2, both BENTO variants increase peak RSS by a negligible 1% on average.

8.4 Runtime Performance and Consistency

Another important metric of a practical defense is its runtime cost. We use two metrics to evaluate performance. *Fuel* is a deterministic

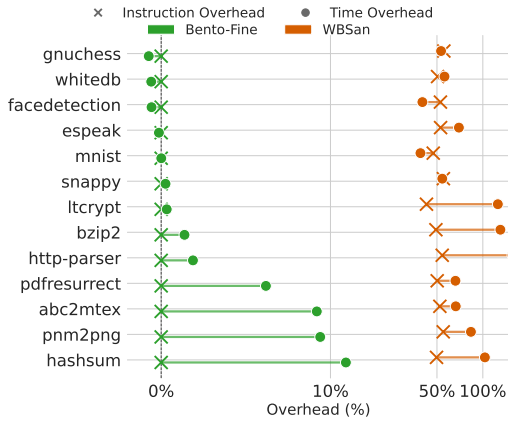


Figure 6: Performance overhead on real-world applications. BENTO-Fine consistently maintains near-zero overhead, while WBSan imposes a significant performance penalty, often exceeding 50% in both instruction count and time.

instruction-counting metric provided by the Wasmtime runtime that serves as a ground-truth measure of computational work. *Wall-clock time* measures real-world performance but is subject to noise from the environment and the Wasm runtime’s JIT compiler.

From a computational standpoint, BENTO is highly efficient. Its computational overhead is zero (1.00x Fuel) across all benchmarks, a direct result of adding no new instructions (Table 2). This fundamental efficiency—adding no new computational work—translates directly to low and predictable real-world performance. BENTO-Coarse adds a 3% average wall-clock slowdown, while Bento-Fine incurs a similarly low 4% average slowdown. As illustrated in Figure 6, the wall-clock overhead for many applications is negligible: *mnist*, *espeak*, and *snappy* show no measurable overhead (0%), while *bzip2* shows only a 1% slowdown. Even for applications with more complex processing like Libpng’s *pnm2png*, the overhead is a modest 9%.

This low and predictable overhead is a direct result of BENTO’s architectural hardening. By embedding policy into the binary’s structure, it avoids the performance penalties associated with injecting runtime checks. This also explains why BENTO avoids the hidden *architectural penalties* common in instrumentation [36, 67]. Figure 6 visualizes this divergence: for some applications, the wall-clock penalty of an instrumentation-based approach is substantially higher than its instruction overhead. For instance, on *bzip2*, a 49% increase in computational work results in a 130% wall-clock slowdown for an instrumented binary. In contrast, BENTO’s near-zero computational cost consistently translates to low real-world overhead because it preserves the program’s original structure.

While a direct, fair comparison to AOT-compiled systems like MS-Wasm [34, 44] is infeasible due to their differing compilation models, MS-Wasm reported an average 20% runtime overhead relative to native execution on polybench benchmarks. On this same suite, BENTO-Fine incurs only a 5% overhead relative to an uninstrumented Wasm baseline (Table 2).

In summary, our evaluation shows that BENTO’s architectural hardening is both correct and practical. It incurs negligible runtime overhead (<4%), imposes minimal startup and memory costs (1%), and operates automatically on stripped COTS binaries where instrumentation-based defenses fail. These results establish that our approach provides a robust and efficient defense for the modern web ecosystem.

9 Related Work

Wasm Security Research. Research in WebAssembly security targets both the runtime and the guest module. One line of work focuses on the runtime’s reliability, using techniques like differential testing and fuzzing (e.g., LWDIFF [74], Waltzz [69], FreeWavm [51]) to find implementation bugs in the sandbox [7, 51, 64, 69, 72, 74, 75]. While this enhances runtime robustness, it does not prevent memory corruption *within* a guest module, a vulnerability that can be used to bypass SFI guarantees [12, 35, 42]. To address this, prior work on module hardening follows two main strategies. The first requires source-level access or specialized environments, such as compiling C/C++ to memory safe Wasm with MS-Wasm [34, 44], sandboxing with RLBox [46], or using hardware protection keys with PKUWA [37] and custom runtimes like Wave [33]. The second strategy instruments binaries with runtime checks to detect memory errors, as seen in tools like WBSan [67] and Fuzzm [36] [12, 24, 25]. As detailed in our problem statement (Section 3), both strategies are impractical for hardening COTS modules due to their reliance on source code, custom runtimes, and invasive instrumentation.

Binary Rewriting for Security. BENTO’s methodology builds on the established field of binary rewriting for security [1, 9, 13–15, 27, 41, 52, 53, 59, 66, 70]. This technique has been successfully applied to COTS binaries to enable fuzzing with sanitizers like RetroWrite [11] and ASan [71], and to perform emergency hotpatching in systems like Kintsugi [41] and AutoPatch [53]. While BENTO also uses static analysis to rewrite COTS binaries, its contribution is distinct from these systems. Instead of adding reactive checks or patching individual bugs, BENTO rewrites the binary’s structure to enforce a proactive, architectural isolation policy.

10 Conclusion and Summary

This paper presented BENTO, a static binary rewriter that hardens COTS Wasm modules against memory corruption by partitioning monolithic memory into isolated regions using the standard Wasm multi-memory feature. Our evaluation demonstrates that this architectural approach is both effective and practical, as it mitigates real-world exploits and preserves program semantics, a result confirmed through differential testing. Because BENTO embeds security policies into the binary’s structure without adding new instructions, it only incurs negligible performance costs (<3%). Thus, our work introduces proactive, architectural hardening as an efficient strategy for securing the Wasm ecosystem without requiring source code or custom runtimes.

Acknowledgments

This work was partially funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) through SFB 1119 – 236615297, project S2.

References

- [1] Martin Abadi et al. “CFI: Principles, implementations, and applications”. In: *Proc. of the ACM Conference on Computer and Communications Security (CCS)*. 2005. doi: 10.1145/1609956.1609960.
- [2] *abc2mtex – Notate tunes stored in ABC notation*. URL: <https://ctan.org/pkg/abc2mtex> (visited on 09/15/2025).
- [3] Florian Bauckholt et al. “WebAssembly as a fuzzing compilation target (registered report)”. In: *Proc. of the ACM International Fuzzing Workshop*. 2024. doi: 10.1145/3678722.3685531.
- [4] Andrea Bittau et al. “Hacking Blind”. In: *Proc. of the IEEE Symposium on Security and Privacy (S&P)*. 2014. doi: 10.1109/SP.2014.22.
- [5] Tiago Brito et al. “Wasmati: An efficient static vulnerability scanner for WebAssembly”. In: *Computers & Security* (2022). doi: 10.1016/j.cose.2022.102745.
- [6] *bzip2: A program and library for lossless, block-sorting data compression*. URL: <https://gitlab.com/federicomenuquintero/bzip2> (visited on 10/22/2025).
- [7] Shangdong Cao et al. “WASMaker: Differential testing of WebAssembly runtimes via semantic-aware binary generation”. In: *Proc. of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2024. doi: 10.1145/3650212.3680358.
- [8] Xingman Chen et al. “MTSan: A feasible and practical memory sanitizer for fuzzing COTS binaries”. In: *USENIX Security Symposium*. 2023.
- [9] Lucas Davi et al. “MoCFI: A Framework to Mitigate Control-Flow Attacks on Smartphones.” In: *Network and Distributed System Security Symposium (NDSS)*. 2012.
- [10] Lucas Davi et al. “Stitching the gadgets: On the ineffectiveness of coarse-grained control-flow integrity protection”. In: *USENIX Security Symposium*. 2014.
- [11] Sushant Dinesh et al. “RetroWrite: Statically Instrumenting COTS Binaries for Fuzzing and Sanitization”. In: *Proc. of the IEEE Symposium on Security and Privacy (S&P)*. 2020. doi: 10.1109/SP40000.2020.00009.
- [12] Oussama Draissi et al. “Wemby’s Web: Hunting for memory corruption in WebAssembly”. In: *Proc. of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2025. doi: 10.1145/3728937.
- [13] Gregory J. Duck et al. “Binary rewriting without control flow recovery”. In: *Proc. of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2020. doi: 10.1145/3385412.3385972.
- [14] Gregory J. Duck et al. “Binary rewriting without control flow recovery”. In: *Proc. of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2020. doi: 10.1145/3385412.3385972.
- [15] Gregory J. Duck et al. “Hardening binaries against more memory errors”. In: *Proc. of the European Conference on Computer Systems (EUROSYS)*. 2022. doi: 10.1145/3492321.3519580.
- [16] *eSpeak NG Text-to-Speech*. URL: <https://github.com/espeak-ng/espeak-ng> (visited on 10/22/2025).
- [17] Chris Fallin et al. “Partial evaluation, whole-program compilation”. In: *Proc. of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. doi: 10.1145/3729259.
- [18] Andrea Fioraldi et al. “Fuzzing Binaries for Memory Safety Errors with QASan”. In: *IEEE Secure Development (SecDev)*. 2020. doi: 10.1109/SecDev45635.2020.00019.
- [19] *GNU Chess*. URL: <https://www.gnu.org/software/chess> (visited on 10/22/2025).
- [20] *Google Chrome Platform Status*. Accessed: 2025-9-19. URL: <https://chromestatus.com/metrics/feature/timeline/popularity/2237>.
- [21] Matthew R Guthaus et al. “MiBench: A free, commercially representative embedded benchmark suite”. In: *Proc. of the fourth annual IEEE international workshop on workload characterization. WWC-4 (Cat. No. 01EX538)*. 2001.
- [22] Andreas Haas et al. “Bringing the web up to speed with WebAssembly”. In: *Proc. of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2017. doi: 10.1145/3062341.3062363.
- [23] Keno Haßler et al. “WAFI: Binary-Only WebAssembly Fuzzing with Fast Snapshots”. In: *Reversing and Offensive-oriented Trends Symposium*. 2021. doi: 10.1145/3503921.3503924.
- [24] Ningyu He et al. “Eunomia: Enabling User-Specified Fine-Grained Search in Symbolically Executing WebAssembly Binaries”. In: *Proc. of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2023. doi: 10.1145/3597926.3598064.
- [25] Ningyu He et al. “SeeWasm: An efficient and fully-functional symbolic execution engine for WebAssembly binaries”. In: *Proc. of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2024. doi: 10.1145/3650212.3685300.
- [26] Ningyu He et al. “The promise and pitfalls of WebAssembly: Perspectives from the industry”. In: *Proc. of the ACM SIGSOFT Symposium on the Foundations of Software Engineering*. 2025. doi: 10.1145/3696630.3728570.
- [27] Yi He et al. “RapidPatch: Firmware Hotpatching for Real-Time Embedded Devices”. In: *USENIX Security Symposium*. 2022.
- [28] Aaron Hilbig et al. “An Empirical Study of Real-World WebAssembly Binaries: Security, Languages, Use Cases”. In: *Proc. of the ACM Web Conference (WWW)*. 2021. doi: 10.1145/3442381.3450138.
- [29] *HTTP Parser*. URL: <https://github.com/nodejs/http-parser> (visited on 10/22/2025).
- [30] Hong Hu et al. “Automatic generation of data-oriented exploits”. In: *USENIX Security Symposium*. 2015.
- [31] Hong Hu et al. “Data-Oriented Programming: On the Expressiveness of Non-control Data Attacks”. In: *Proc. of the IEEE Symposium on Security and Privacy (S&P)*. 2016. doi: 10.1109/SP.2016.62.
- [32] *JetStream 2.2*. URL: <https://browserbench.org/JetStream/in-depth.html> (visited on 09/13/2025).
- [33] Evan Johnson et al. “WaVe: a verifiably secure WebAssembly sandboxing runtime”. In: *Proc. of the IEEE Symposium on Security and Privacy (S&P)*. 2023. doi: 10.1109/SP46215.2023.10179357.
- [34] Maxime Legoupil et al. “Iris-MSWasm: Elucidating and mechanising the security invariants of memory-safe WebAssembly”. In: *Proceedings of the ACM on Programming Languages OOPSLA2* (2024).
- [35] Daniel Lehmann et al. “Everything Old is New Again: Binary Security of WebAssembly”. In: *USENIX Security Symposium*. 2020.
- [36] Daniel Lehmann et al. “Fuzzm: Finding Memory Bugs through Binary-Only Instrumentation and Fuzzing of WebAssembly”. In: (2021). arXiv: 2110.15433 [cs.CR].
- [37] Hanwen Lei et al. “Put your memory in order: Efficient domain-based memory isolation for WASM applications”. In: *Proc. of the ACM Conference on Computer and Communications Security (CCS)*. 2023. doi: 10.1145/3576915.3623205.
- [38] *libfacedetection: An open source library for face detection in images. The face detection speed can reach 1000FPS*. URL: <https://github.com/ShiqiYu/libfacedetection> (visited on 09/14/2025).
- [39] *libpng Home Page*. URL: <https://www.libpng.org/pub/png/libpng.html> (visited on 09/15/2025).
- [40] *libtomcrypt: Cryptographic Toolkit*. URL: <https://github.com/libtom/libtomcrypt> (visited on 09/15/2025).
- [41] Philipp Mackensen et al. “KINTSUGI: secure hotpatching for code-shadowing real-time embedded systems”. In: *USENIX Security Symposium*. 2025.
- [42] Brian McFadden et al. “Security chasms of wasm”. In: *Proc. of Black Hat USA* (2018).

- [43] *Memory Protection Keys, The Linux Kernel documentation*. URL: <https://docs.kernel.org/core-api/protection-keys.html> (visited on 09/23/2025).
- [44] Alexandra E Michael et al. “MSWasm: Soundly enforcing memory-safe execution of unsafe code”. In: *Proc. of the ACM on Programming Languages (POPL)*. DOI: 10.1145/3571208.
- [45] *MNIST Neural Network*. URL: <https://github.com/AndrewCarterUK/mnist-neural-network-plain-c> (visited on 10/22/2025).
- [46] Shravan Narayan et al. “Retrofitting fine grain isolation in the Firefox renderer”. In: *USENIX Security Symposium*. 2020.
- [47] Nicholas Nethercote et al. “Valgrind: a framework for heavyweight dynamic binary instrumentation”. In: *Proc. of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2007. DOI: 10.1145/1250734.1250746.
- [48] Aleph One. “Smashing the stack for fun and profit”. In: *Phrack magazine* (1996).
- [49] *PDFResurrect is a tool aimed at analyzing PDF documents*. URL: <https://github.com/enferex/pdfresurrect> (visited on 10/22/2025).
- [50] Louis-Noël Pouchet et al. *The polyhedral benchmark suite*. 2012. URL: <http://www.cse.ohiostate.edu/pouchet/software/polybench>.
- [51] Peng Qian et al. “FreeWasm: Enhanced WebAssembly Runtime Fuzzing Guided by Parse Tree Mutation and Snapshot”. In: *Proc. of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2025. DOI: 10.1145/3728877.
- [52] Prashant Hari Narayan Rajput et al. “ICSPatch: Automated Vulnerability Localization and Non-Intrusive Hotpatching in Industrial Control Systems using Data Dependence Graphs”. In: *USENIX Security Symposium*. 2023.
- [53] Mohsen Salehi et al. “AutoPatch: Automated Generation of Hotpatches for Real-Time Embedded Devices”. In: *Proc. of the ACM Conference on Computer and Communications Security (CCS)*. 2024. DOI: 10.1145/3658644.3690255.
- [54] Konstantin Serebryany et al. “AddressSanitizer: a fast address sanity checker”. In: *Proc. of the USENIX Annual Technical Conference (ATC)*. 2012. DOI: 10.5555/2342821.2342849.
- [55] Hovav Shacham et al. “The geometry of innocent flesh on the bone: return-into-libc without function calls (on the x86)”. In: *Proc. of the ACM Conference on Computer and Communications Security (CCS)*. 2007. DOI: 10.1145/1315245.1315313.
- [56] *Snappy, a fast compressor/decompressor*. URL: <https://github.com/google/snappy> (visited on 10/22/2025).
- [57] PaX Team. *PaX Address Space Layout Randomization (ASLR)*. 2001. URL: <https://pax.grsecurity.net/docs>.
- [58] *Toolchain support for multiple memories*. GitHub. Aug. 4, 2023. URL: <https://github.com/WebAssembly/multi-memory/issues/45> (visited on 10/22/2025).
- [59] Victor van der Veen et al. “Practical Context-Sensitive CFI”. In: *Proc. of the ACM Conference on Computer and Communications Security (CCS)*. 2015. DOI: 10.1145/2810103.2813673.
- [60] Robert Wahbe et al. “Efficient software-based fault isolation”. In: *Proceedings of the fourteenth ACM symposium on Operating systems principles*. 1993. DOI: 10.1145/168619.168635.
- [61] Wenwen Wang. “How far we’ve come – A characterization study of standalone WebAssembly runtimes”. In: *IEEE International Symposium on Workload Characterization (IISWC)*. 2022. DOI: 10.1109/iiswc55918.2022.00028.
- [62] *Wasm 3.0*. URL: <https://webassembly.org/news/2025-09-17-wasm-3.0/> (visited on 09/18/2025).
- [63] *Wasm Security*. Accessed: 2024-09-18. URL: <https://webassembly.org/docs/security/>.
- [64] *wasm-smith: CLI and Rust libraries for low-level manipulation of WebAssembly modules*. URL: <https://github.com/bytecodealliance/wasm-tools/tree/main/crates/wasm-smith> (visited on 09/01/2025).
- [65] *WhiteDB: An in-memory database library written in C*. URL: <https://github.com/priitj/whitedb> (visited on 10/22/2025).
- [66] Jan Wichelmann et al. “Cipherfix: Mitigating Ciphertext Side-Channel Attacks in Software”. In: *USENIX Security Symposium*. 2023.
- [67] Xiao Wu et al. “WBSan: WebAssembly bug detection for sanitization and binary-only fuzzing”. In: *Proc. of the ACM Web Conference (WWW)*. 2025. DOI: 10.1145/3696410.3714622.
- [68] Xuejun Yang et al. “Finding and understanding bugs in C compilers”. In: *Proc. of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2011. DOI: 10.1145/1993498.1993532.
- [69] Lingming Zhang et al. “Waltzz: WebAssembly Runtime Fuzzing with Stack-Invariant Transformation”. In: *USENIX Security Symposium*. 2025.
- [70] Mingwei Zhang et al. “Control flow integrity for COTS binaries”. In: *USENIX Security Symposium*. 2013.
- [71] Yuchen Zhang et al. “Debloating Address Sanitizer”. In: *USENIX Security Symposium*. 2022.
- [72] Wenxuan Zhao et al. “Waplique: Testing WebAssembly Runtime via Execution Context-Aware Bytecode Mutation”. In: *Proc. of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2024. DOI: 10.1145/3650212.3680340.
- [73] Wenlong Zheng et al. “WASMDYPA: Effectively detecting WebAssembly bugs via dynamic program analysis”. In: *IEEE International Conference on Software Analysis, Evolution and Reengineering*. 2024. DOI: 10.1109/saner60148.2024.00037.
- [74] Shiyao Zhou et al. “LWDIFF: an LLM-Assisted Differential Testing Framework for Webassembly Runtimes”. In: *Proc. of the International Conference on Software Engineering (ICSE)*. 2025. DOI: 10.1109/ICSE55347.2025.00233.
- [75] Shiyao Zhou et al. “WADIFF: A Differential Testing Framework for WebAssembly Runtimes”. In: *Proc. of the International Conference on Automated Software Engineering (ASE)*. 2023. DOI: 10.1109/ASE56229.2023.00188.

A Evaluation Details

In this appendix, we provide additional details about our evaluation methodology and results.

Table 3: Benchmark suites included in our evaluation. This table is primarily based on the original WABench suite [61]. Workloads for all programs in these suites are the default configurations provided by their respective original benchmark suites. Real-world applications are detailed in Table 4.

	Benchmark Program	Application Domain	Description
<i>JetStream2</i> [32]	gcc-loops	Compilation	Loops used to tune GCC vectorizer
	hashset	Hash table	Hash table operations of web page loading
	quicksort	Data Sorting	Quick sort algorithm implementation
	tsf	Data processing	Implementation of a typed stream format
<i>MiBench</i> [21]	basicmath	Automotive	Basic mathematical computations
	bitcount	Automotive	Bit manipulations
	cjpeg	Consumer multimedia	JPEG image compression/decompression
	djpeg	Consumer multimedia	JPEG image compression/decompression
	stringsearch	Office automation	Searching given words in phrases
	blowfish	Security	Symmetric block cipher
	rijndael	Security	Block cipher with variable length keys
	sha	Security	Secure hash algorithm
	crc32	Telecommunications	32-Bit Cyclic Redundancy Check
	rawaudio	Audio processing	Encoding a large PCM audio file
	rawdaudio	Audio processing	Decoding a large ADPCM audio file
<i>PolyBench</i> [50]	correlation	Data mining	Correlation computation
	covariance	Data mining	Covariance computation
	gemm	Linear algebra	Matrix multiplication
	gemver	Linear algebra	Vector multiplication and matrix addition
	gesummv	Linear algebra	Scalar, vector and matrix multiplication
	symm	Linear algebra	Symmetric matrix multiplication
	syr2k	Linear algebra	Symmetric rank-2k operations
	syrk	Linear algebra	Symmetric rank-k operations
	trmm	Linear algebra	Triangular matrix multiplication
	2mm	Linear algebra	Two matrix multiplications
	3mm	Linear algebra	Three matrix multiplications
	atax	Linear algebra	Matrix transpose and vector multiplication
	bicg	Linear algebra	BiCG sub kernel of BiCGStab linear solver
	doitgen	Linear algebra	Multiresolution analysis kernel
	mvt	Linear algebra	Matrix vector product and transpose
	cholesky	Linear algebra	Cholesky decomposition
	durbin	Linear algebra	Toeplitz system solver
	gramschmidt	Linear algebra	Gram-Schmidt
	lu	Linear algebra	LU decomposition
	ludcmp	Linear algebra	LU decomposition
	trisolv	Linear algebra	Triangular solver
	deriche	Image processing	Edge detection filter
	floyd-warshall	Physics simulation	Computing shortest paths in a graph
	nussinov	Dynamic programming	Sequence alignment
	adi	Systems control	Alternating direction implicit solver
	fdtd-2d	Physics simulation	2-D finite-difference time-domain kernel
	heat-3d	Physics simulation	Heat equation over 3D data domain
	jacobi-1d	Linear algebra	1-D jacobi stencil computation
	jacobi-2d	Linear algebra	2-D jacobi stencil computation
	seidel-2d	Linear algebra	2-D seidel stencil computation

Table 4: Real-world applications used in our evaluation, supplementing the benchmark suites from Table 3.

Benchmark Program	Application Domain	Description	Workload
bzip2 [6]	File management	File compression/decompression	Compressing a 120MB data file
espeak [16]	NLP	Text-to-Speech synthesizer	Reading out a 200K text file
facedetection [38]	Computer vision	Detecting human faces in images	Detecting faces in a photo
gnuchess [19]	Gaming	Chess-playing game	Playing a single round game (depth 10)
mnist [45]	Machine learning	A neural network for digit recognition	Training the network for 1000 iterations
snappy [56]	Big data processing	Data compression/decompression library	Compressing 512MB memory data
whitedb [65]	Database	Lightweight NoSQL database	Conducting a set of database operations
http-parser [29]	Networking	URL parsing library	Parsing a complex URL
pdfresurrect [49]	File recovery	PDF structure analysis tool	Analyzing a sample PDF file
libpng [39]	Image conversion	PNM to PNG image converter	Converting a sample PNM image
abc2mtex [2]	Typesetting	ABC music notation to TeX converter	Processing a sample ABC file
hashsum [40]	Security	Cryptographic hashsum utility	Calculating RMD-320 hash of a file
ltdcrypt [40]	Security	Cryptographic toolkit utility	Encrypting a file using AES

Table 5: Detailed static overheads relative to an uninstrumented Wasm module. All results are geometric means of overhead ratios. (Sz=File Size, Op=Instructions, Br=Branch Instructions).

Workload	B-Coarse		B-Fine		WBSan			ASan		
	Sz	Op&Br	Sz	Op&Br	Sz	Op	Br	Sz	Op	Br
Overall	1.04	1.00	1.06	1.00	1.51	1.67	2.48	10.16	1.68	3.93
polybench	1.02	1.00	1.02	1.00	1.27	1.40	2.06	13.83	1.46	4.06
mibench	1.06	1.00	1.07	1.00	1.52	1.71	2.64	11.39	2.31	6.00
jetstream	1.04	1.00	1.09	1.00	2.03	2.24	3.21	4.10	1.50	2.61
abc2mtex	1.04	1.00	1.05	1.00	1.88	2.08	2.87	2.69	2.80	2.64
bzip2	1.06	1.00	1.10	1.00	1.92	2.12	2.99	4.03	4.50	4.17
espeak	1.05	1.00	1.10	1.00	2.23	2.46	3.43	2.64	2.73	2.33
facedetection	1.16	1.00	1.19	1.00	2.00	2.22	3.17	2.39	2.74	2.05
gnuchess	1.05	1.00	1.09	1.00	1.97	2.18	3.07	2.98	3.14	2.87
hashsum	1.14	1.00	1.25	1.00	1.90	2.12	3.50	6.39	1.65	4.51
ltdcrypt	1.09	1.00	1.23	1.00	2.06	2.30	3.79	6.35	1.64	4.21
mnist	1.09	1.00	1.09	1.00	2.19	2.43	3.75	12.23	1.39	3.51
pdfresurrect	1.06	1.00	1.07	1.00	2.04	2.26	3.39	10.30	1.13	2.49
pnm2png	1.14	1.00	1.19	1.00	2.20	2.44	3.41	7.42	1.04	1.90
snappy	1.13	1.00	1.23	1.00	2.14	2.38	3.60	13.76	1.32	3.35
http-parser	1.06	1.00	1.08	1.00	2.02	2.24	3.18	13.16	1.36	2.83
whitedb	1.05	1.00	1.06	1.00	1.87	2.07	3.06	9.19	1.10	2.32